

02 — Classical Machine Learning

Time: ~4-5 hours | **Level:** Beginner → Intermediate

What you'll learn:

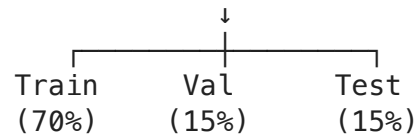
- The ML pipeline: train/validate/test splits
- Linear Regression: predicting numbers
- Logistic Regression: predicting categories
- Decision Trees & Random Forests
- Evaluation metrics: accuracy is NOT enough
- The bias-variance tradeoff (most important concept in ML)
- Cross-validation: robust model evaluation

Prerequisites: Notebook 01 (NumPy, Pandas, basic statistics)

1. The ML Pipeline

Every ML project follows this structure:

Raw Data → Clean/Feature Engineer → Split → Train Model → Evaluate → Deploy



Why Three Splits?

Split	Purpose	When To Use
Train	Model learns patterns from this data	During training
Validation	Tune hyperparameters, catch overfitting	During development
Test	Final unbiased evaluation	Once, at the very end

⚠ The Cardinal Rule

Never let the model see test data during training. This is called **data leakage** and gives you fake (optimistic) performance numbers.

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```

import seaborn as sns
from sklearn.model_selection import train_test_split

sns.set_theme(style='whitegrid')
np.random.seed(42)

# — Generate synthetic patient data —————
n = 1000
age = np.random.randint(18, 75, n)
exercise = np.random.randint(0, 7, n)
sleep = np.random.normal(7, 1.5, n).clip(3, 11).round(1)
therapy = np.random.choice([0, 1], n, p=[0.6, 0.4])

# Depression score (what we want to predict) – influenced by other features
depression = (
    15
    - 0.05 * age
    - 0.8 * exercise
    - 0.5 * sleep
    - 2.0 * therapy
    + np.random.normal(0, 2.5, n) # noise
).clip(0, 20).round(1)

# High risk label (binary classification target)
high_risk = (depression > 12).astype(int)

df = pd.DataFrame({
    'age': age, 'exercise': exercise, 'sleep_hours': sleep,
    'therapy': therapy, 'depression_score': depression,
    'high_risk': high_risk,
})

print(f"Dataset: {n} patients")
print(f"High risk: {high_risk.sum()} ({high_risk.mean()*100:.1f}%)")
df.head()

```

```

In [ ]: # — Train / Validation / Test Split —————

# Features (X) and targets (y)
features = ['age', 'exercise', 'sleep_hours', 'therapy']
X = df[features]
y_regression = df['depression_score'] # for regression
y_classification = df['high_risk']    # for classification

# Step 1: Split off test set (15%)
X_temp, X_test, y_reg_temp, y_reg_test, y_cls_temp, y_cls_test = train_test_
    X, y_regression, y_classification, test_size=0.15, random_state=42
)

# Step 2: Split remaining into train (70%) and validation (15%)
X_train, X_val, y_reg_train, y_reg_val, y_cls_train, y_cls_val = train_test_
    X_temp, y_reg_temp, y_cls_temp, test_size=0.176, random_state=42 # 0.17
)

```

```
print(f"Train: {len(X_train)} | Val: {len(X_val)} | Test: {len(X_test)}")
print(f"Total: {len(X_train) + len(X_val) + len(X_test)} (should be {n})")
```

2. Linear Regression — Predicting Numbers

Problem: Given patient features (age, exercise, sleep, therapy), predict their depression score.

Linear regression finds the best straight line (or hyperplane in higher dimensions):

$$\hat{y} = w_1 \cdot \text{age} + w_2 \cdot \text{exercise} + w_3 \cdot \text{sleep} + w_4 \cdot \text{therapy} + b$$

Where:

- w_1, w_2, w_3, w_4 are **weights** (learned from data)
- b is the **bias** (intercept)
- $\hat{y}$ is the **prediction**

The model finds weights that **minimise the error** between predictions and actual values.

```
In [ ]: from sklearn.linear_model import LinearRegression
        from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

        # — Train a linear regression model —
        lr_model = LinearRegression()
        lr_model.fit(X_train, y_reg_train) # Learn weights from training data

        # What did it learn?
        print("Learned weights:")
        for feature, weight in zip(features, lr_model.coef_):
            direction = "↑ depression" if weight > 0 else "↓ depression"
            print(f" {feature:15s}: {weight:+.4f} (each unit increase → {direction})")
        print(f" {'bias':15s}: {lr_model.intercept_:+.4f}")

        # — Evaluate on validation set —
        y_pred_val = lr_model.predict(X_val)

        mse = mean_squared_error(y_reg_val, y_pred_val)
        rmse = np.sqrt(mse)
        mae = mean_absolute_error(y_reg_val, y_pred_val)
        r2 = r2_score(y_reg_val, y_pred_val)

        print(f"\n📊 Validation Metrics:")
        print(f" RMSE: {rmse:.3f} (average error in score units)")
        print(f" MAE: {mae:.3f} (average absolute error)")
        print(f" R²: {r2:.3f} (1.0 = perfect, 0.0 = useless)")
```

```
In [ ]: # — Visualise: Predicted vs Actual —
        fig, axes = plt.subplots(1, 2, figsize=(14, 5))

        # Scatter: predicted vs actual
        axes[0].scatter(y_reg_val, y_pred_val, alpha=0.4, s=20)
```

```

axes[0].plot([0, 20], [0, 20], 'r--', linewidth=2, label='Perfect prediction')
axes[0].set_xlabel('Actual Depression Score')
axes[0].set_ylabel('Predicted Depression Score')
axes[0].set_title(f'Linear Regression: Predicted vs Actual (R²={r2:.3f})')
axes[0].legend()

# Residuals: errors should be randomly distributed around 0
residuals = y_reg_val - y_pred_val
axes[1].hist(residuals, bins=30, color='steelblue', alpha=0.7, edgecolor='b')
axes[1].axvline(0, color='red', linestyle='--')
axes[1].set_xlabel('Prediction Error (Actual - Predicted)')
axes[1].set_ylabel('Frequency')
axes[1].set_title('Residual Distribution (should be centered at 0)')

plt.tight_layout()
plt.show()

```

3. Logistic Regression — Predicting Categories

Problem: Given patient features, predict whether they are **high-risk** (1) or **not** (0).

Logistic regression is linear regression + a **sigmoid function** that squashes output to [0, 1]:

$$P(\text{high risk}) = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

If $P > 0.5$, predict "high risk". Otherwise, predict "not high risk".

The Sigmoid Function

```

In [ ]: # — Visualise the sigmoid function —————
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z = np.linspace(-8, 8, 200)
plt.figure(figsize=(8, 4))
plt.plot(z, sigmoid(z), linewidth=2, color='coral')
plt.axhline(0.5, color='gray', linestyle='--', alpha=0.5)
plt.axvline(0, color='gray', linestyle='--', alpha=0.5)
plt.xlabel('z = wx + b (linear output)')
plt.ylabel('σ(z) = probability')
plt.title('Sigmoid Function: Maps any number → probability [0, 1]')
plt.annotate('If σ(z) > 0.5 → predict class 1', xy=(2, 0.88), fontsize=11, c='red')
plt.annotate('If σ(z) < 0.5 → predict class 0', xy=(-7.5, 0.12), fontsize=11, c='red')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

```

In [ ]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_repor

```

```
# — Train —————
log_model = LogisticRegression(random_state=42)
log_model.fit(X_train, y_cls_train)

# — Predict on validation set —————
y_pred_cls = log_model.predict(X_val)

print("📊 Classification Report:")
print(classification_report(y_cls_val, y_pred_cls, target_names=['Low Risk',
```

4. Why Accuracy Is NOT Enough

Imagine a dataset where only 5% of patients are high-risk. A model that **always says "low risk"** gets 95% accuracy! But it's completely useless.

The Confusion Matrix

		Predicted		
		Low Risk	High Risk	
Actual	Low Risk	TN	FP	← False Positive:
	High Risk	FN	TP	← False Negative:
		MISSED a real case		

Metric	Formula	What It Answers
Precision	$TP / (TP + FP)$	"Of those we flagged, how many were actually high-risk?"
Recall	$TP / (TP + FN)$	"Of all actual high-risk patients, how many did we catch?"
F1 Score	$2 \times (P \times R) / (P + R)$	Harmonic mean of precision and recall

In Healthcare: Recall > Precision

Missing a high-risk patient (False Negative) is **much worse** than a false alarm (False Positive).

```
In [ ]: # — Confusion Matrix Visualisation —————
cm = confusion_matrix(y_cls_val, y_pred_cls)

fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
            xticklabels=['Low Risk', 'High Risk'],
            yticklabels=['Low Risk', 'High Risk'])
ax.set_xlabel('Predicted')
ax.set_ylabel('Actual')
ax.set_title('Confusion Matrix')
plt.tight_layout()
plt.show()
```

```
tn, fp, fn, tp = cm.ravel()
print(f"True Negatives: {tn} (correctly identified low-risk)")
print(f"False Positives: {fp} (false alarms – said high-risk but wasn't)")
print(f"False Negatives: {fn} (MISSED high-risk patients 🚨)")
print(f"True Positives: {tp} (correctly identified high-risk)")
print(f"\n💡 In healthcare, we want to MINIMISE False Negatives (FN={fn}).")
```

5. Decision Trees & Random Forests

Decision Tree: Learns a series of if/else rules from data.

```
Is exercise < 2?
├── Yes: Is sleep_hours < 5?
│   ├── Yes → High Risk (90%)
│   └── No → Moderate Risk (60%)
└── No: Is therapy = 1?
    ├── Yes → Low Risk (85%)
    └── No → Moderate Risk (45%)
```

Random Forest: Builds many trees (e.g., 100) on random subsets of data, then **votes** on the answer. This reduces overfitting dramatically.

```
In [ ]: from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# — Decision Tree —
tree = DecisionTreeClassifier(max_depth=4, random_state=42)
tree.fit(X_train, y_cls_train)
tree_pred = tree.predict(X_val)

# — Random Forest —
forest = RandomForestClassifier(n_estimators=100, max_depth=6, random_state=
forest.fit(X_train, y_cls_train)
forest_pred = forest.predict(X_val)

# — Compare —
print("Model Comparison (Validation Set):")
print(f"{'Model':<25} {'Accuracy':>10} {'F1':>10} {'Recall':>10}")
print("-" * 55)
for name, pred in [('Logistic Regression', y_pred_cls),
                   ('Decision Tree', tree_pred),
                   ('Random Forest', forest_pred)]:
    acc = accuracy_score(y_cls_val, pred)
    f1 = f1_score(y_cls_val, pred)
    rec = recall_score(y_cls_val, pred)
    print(f"{'name':<25} {'acc':>10.3f} {'f1':>10.3f} {'rec':>10.3f}")
```

```
In [ ]: # — Feature Importance (what the model thinks matters) —
importance = pd.Series(forest.feature_importances_, index=features).sort_val

plt.figure(figsize=(8, 4))
importance.plot(kind='barh', color='steelblue')
```

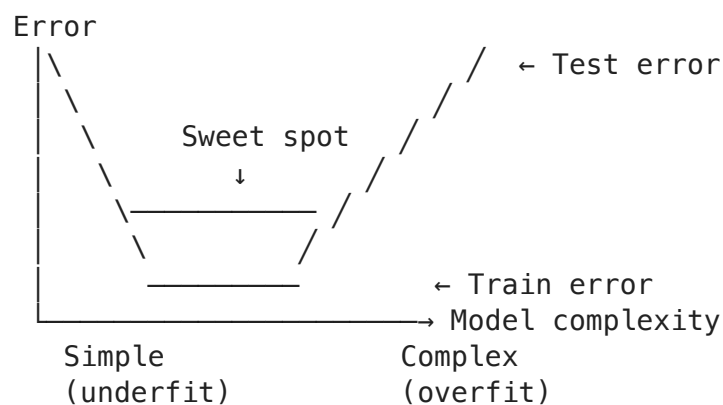
```
plt.xlabel('Feature Importance (higher = more influential)')
plt.title('Random Forest: What Drives High-Risk Predictions?')
plt.tight_layout()
plt.show()

print("💡 The model learned that exercise and sleep matter most – matches our intuition")
```

6. The Bias-Variance Tradeoff

The most important concept in all of ML.

Concept	Meaning	Symptom
High Bias (Underfitting)	Model is too simple; can't capture patterns	Low train accuracy, low test accuracy
High Variance (Overfitting)	Model memorised training data; fails on new data	High train accuracy, low test accuracy
Sweet Spot	Good generalisation	High train accuracy, high test accuracy



```
In [ ]: # — Demonstrate overfitting with tree depth —————
depths = range(1, 25)
train_scores, val_scores = [], []

for d in depths:
    tree = DecisionTreeClassifier(max_depth=d, random_state=42)
    tree.fit(X_train, y_cls_train)
    train_scores.append(tree.score(X_train, y_cls_train))
    val_scores.append(tree.score(X_val, y_cls_val))

plt.figure(figsize=(10, 5))
plt.plot(depths, train_scores, 'b-o', label='Training accuracy', markersize=10)
plt.plot(depths, val_scores, 'r-o', label='Validation accuracy', markersize=10)
plt.axvline(x=depths[np.argmax(val_scores)], color='green', linestyle='--',
            label=f'Best depth = {depths[np.argmax(val_scores)]}')
plt.xlabel('Tree Depth (model complexity →)')
plt.ylabel('Accuracy')
plt.title('Bias-Variance Tradeoff: Training vs Validation Accuracy')
plt.legend()
```

```
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("💡 Notice: Training accuracy keeps rising, but validation peaks then
print(" The gap between them is OVERFITTING.")
print(f" Best depth: {depths[np.argmax(val_scores)]} (validation accuracy:"))
```

7. Cross-Validation — Robust Evaluation

A single train/val split can be "lucky" or "unlucky". **K-Fold Cross-Validation** trains K different models, each with a different validation fold:

```
Fold 1: [VAL ] [Train] [Train] [Train] [Train]
Fold 2: [Train] [VAL ] [Train] [Train] [Train]
Fold 3: [Train] [Train] [VAL ] [Train] [Train]
Fold 4: [Train] [Train] [Train] [VAL ] [Train]
Fold 5: [Train] [Train] [Train] [Train] [VAL ]
```

Final score = average across all folds. More reliable than a single number.

```
In [ ]: from sklearn.model_selection import cross_val_score

models = {
    'Logistic Regression': LogisticRegression(random_state=42),
    'Decision Tree (d=5)': DecisionTreeClassifier(max_depth=5, random_state=
    'Random Forest': RandomForestClassifier(n_estimators=100, max_depth=6, r
}

print("5-Fold Cross-Validation Results:")
print(f"{'Model':<25} {'Mean F1':>10} {'Std':>10} {'Individual Folds'}")
print("-" * 75)

for name, model in models.items():
    scores = cross_val_score(model, X_train, y_cls_train, cv=5, scoring='f1'
    fold_str = ', '.join(f'{s:.3f}' for s in scores)
    print(f"{'name':<25} {scores.mean():>10.3f} {scores.std():>10.3f} [{fold_s

print("\n💡 If std is high, the model's performance is unstable across data
```

Exercises

1. Add a new feature `age_squared = age**2` and retrain the linear regression. Does R^2 improve?
2. Train a Random Forest with `n_estimators=500`. Does cross-validation F1 change significantly compared to 100?
3. Plot the ROC curve for the logistic regression model (use `sklearn.metrics.roc_curve`).
4. Try `class_weight='balanced'` in LogisticRegression. How does it affect recall?

✓ Key Takeaways

1. **Always split data** into train/val/test before doing anything
2. **Linear Regression** predicts numbers; **Logistic Regression** predicts categories
3. **Accuracy is misleading** — use F1, precision, recall (especially in imbalanced data)
4. **Overfitting** = memorising training data. Use validation to detect it.
5. **Random Forests** are strong baselines that are hard to beat without deep learning
6. **Cross-validation** gives more reliable estimates than a single split

Why This Matters for LLMs

Every concept here applies to LLM fine-tuning:

- Train/val/test splits → same for fine-tuning
- Overfitting → LLMs overfit even faster (billions of parameters!)
- Metrics → we use F1 for classification + ROUGE for generation
- Cross-validation → we use eval_loss during training

Next: [03 — Neural Networks from Scratch](#)